

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    roc_curve,
    auc
)
```

```
df = pd.read_csv('/content/StudentsPerformance.csv')
```

```
df.head()
```

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score
0	female	group B	bachelor's degree	standard	none	72	72	74
1	female	group C	some college	standard	completed	69	90	88
2	female	group B	master's degree	standard	none	90	95	93
3	male	group A	associate's degree	free/reduced	none	47	57	44

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 8 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   gender                                1000 non-null   object
1   race/ethnicity                        1000 non-null   object
2   parental level of education           1000 non-null   object
3   lunch                                 1000 non-null   object
4   test preparation course               1000 non-null   object
5   math score                           1000 non-null   int64
6   reading score                        1000 non-null   int64
7   writing score                         1000 non-null   int64
dtypes: int64(3), object(5)
memory usage: 62.6+ KB
```

```
df.describe()
```

	math score	reading score	writing score
count	1000.00000	1000.00000	1000.00000
mean	66.08900	69.169000	68.054000
std	15.16308	14.600192	15.195657
min	0.00000	17.000000	10.000000
25%	57.00000	59.000000	57.750000
50%	66.00000	70.000000	69.000000
75%	77.00000	79.000000	79.000000
max	100.00000	100.000000	100.000000

```
df.isnull().sum()
```

```

0
gender      0
race/ethnicity 0
parental level of education 0
lunch       0
test preparation course 0
math score  0
reading score 0
writing score 0

```

dtype: int64

```

df['average_score'] = (
    df['math score'] +
    df['reading score'] +
    df['writing score']
) / 3

df['result'] = np.where(df['average_score'] >= 50, 1, 0)

df.head()

```

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score	average_score	result
0	female	group B	bachelor's degree	standard	none	72	72	74	72.666667	1
1	female	group C	some college	standard	completed	69	90	88	82.333333	1
2	female	group B	master's degree	standard	none	90	95	93	92.666667	1

Next steps: [Generate code with df](#) [New interactive sheet](#)

```

le = LabelEncoder()

for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])

```

```

X = df.drop(['average_score', 'result'], axis=1)

y = df['result']

```

```

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

```

```

dt_model = DecisionTreeClassifier()

dt_model.fit(X_train, y_train)

dt_pred = dt_model.predict(X_test)

dt_accuracy = accuracy_score(y_test, dt_pred)

print("Decision Tree Accuracy:", dt_accuracy)

```

Decision Tree Accuracy: 0.985

```

rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf_model.fit(X_train, y_train)

rf_pred = rf_model.predict(X_test)

```

```
rf_accuracy = accuracy_score(y_test, rf_pred)

print("Random Forest Accuracy:", rf_accuracy)
```

Random Forest Accuracy: 0.995

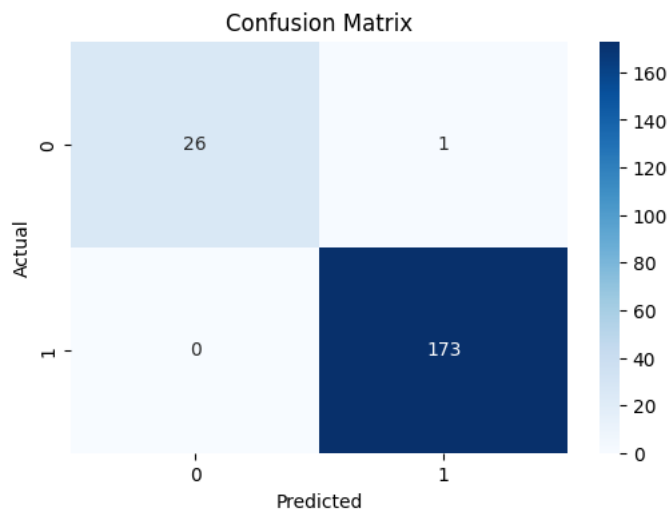
```
cm = confusion_matrix(y_test, rf_pred)

plt.figure(figsize=(6,4))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```



```
print(classification_report(y_test, rf_pred))
```

	precision	recall	f1-score	support
0	1.00	0.96	0.98	27
1	0.99	1.00	1.00	173
accuracy			0.99	200
macro avg	1.00	0.98	0.99	200
weighted avg	1.00	0.99	0.99	200

```
y_prob = rf_model.predict_proba(X_test)[: ,1]

fpr, tpr, thresholds = roc_curve(y_test, y_prob)

roc_auc = auc(fpr, tpr)

plt.figure(figsize=(8,5))

plt.plot(fpr, tpr, label=f'AUC = {roc_auc:.2f}')

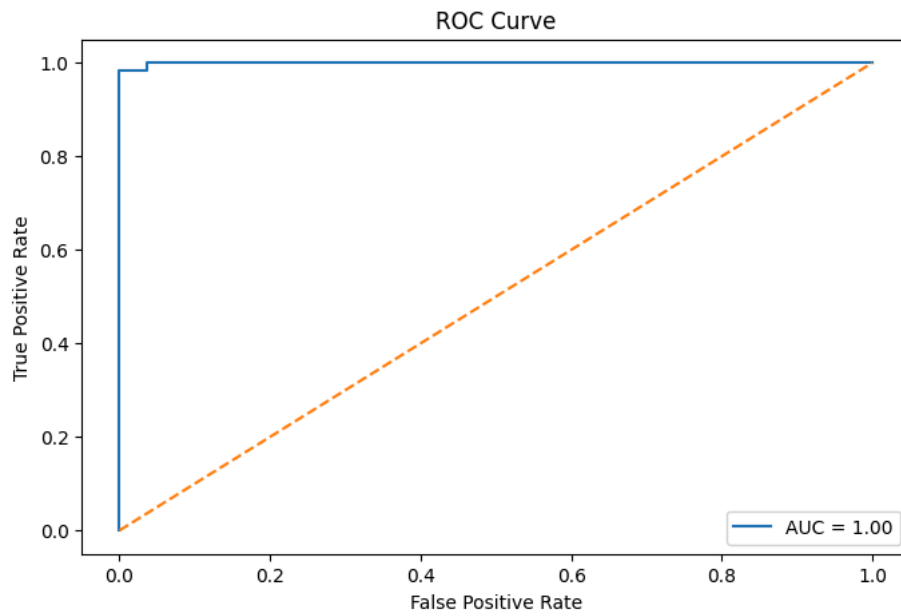
plt.plot([0,1],[0,1], '--')

plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')

plt.title('ROC Curve')

plt.legend()

plt.show()
```



Start coding or [generate](#) with AI.